

Traitement d'images & vision par ordinateur

II. Paramètres de formes

Michèle Gouiffès et Julie Halbout

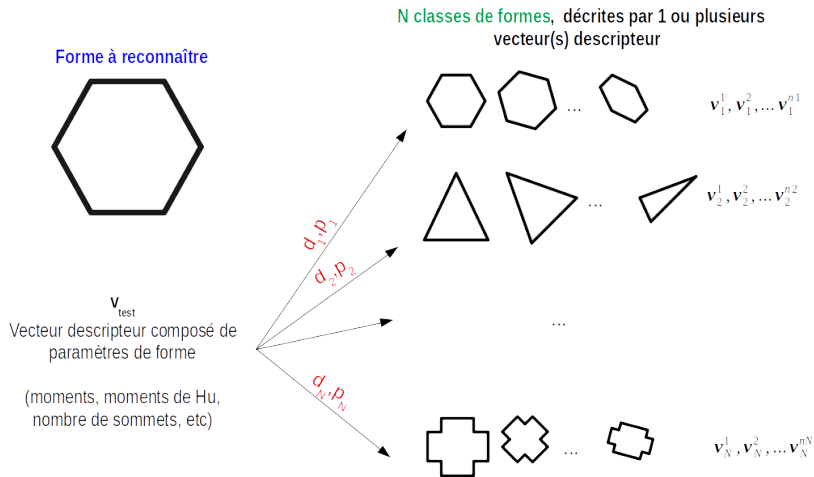
julie.halbout@lisn.fr

université
PARIS-SACLAY



POLYTECH^{*}
PARIS-SACLAY

La reconnaissance de formes

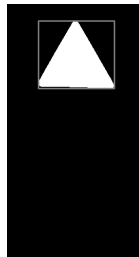
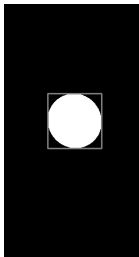


Recherche de la classe :

- permettant de minimiser une distance entre vecteurs d
- ou ayant le maximum de probabilité p

Descripteurs simples

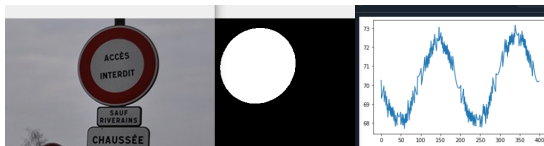
- Aire A_o , périmètre P , centre de gravité $\mathbf{c} = (x_c, y_c)$
- Hauteur H et largeur L du rectangle exinscrit A_r
- Rectangularité : $R = A_o/A_r$
- Compacité $C = \frac{4\pi A_o}{P^2}$ (=1 pour des ronds)
- Nombre d'angles droits, nombre de coins saillants ou entrants.
- Eccentricité : longueur axe majeur/ axe mineur
- Nombre d'Euler : nb. composantes connexes - nb trous



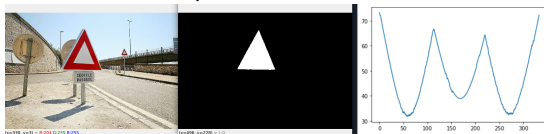
Signature du contour

Soit une forme de centre $\mathbf{c} = (x_c, y_c)$. Pour chaque point de contour $\mathbf{p}_i$, la signature donne la distance euclidienne de chaque $\mathbf{p}_i$ par rapport à $\mathbf{c}$:

$$S(i) = \text{dist}(\mathbf{p}_i, \mathbf{c})$$



Ellipse : 2 max locaux. Cercle : faible variance de $\mathcal{S}$

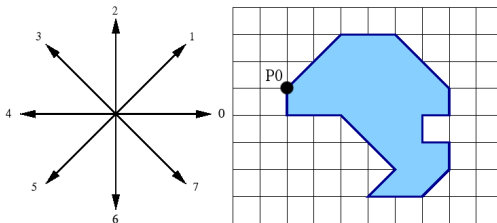


Triangle : trois maxima locaux

Signature de Fourier = FFT de la signature

Codage de Freeman 1/2

- Coder le contour en ne codant que les déplacements relatifs.
- En chaque point du contour, il y a soit 4 directions (en 4-connexité), soit 8 directions (8-connexité) possibles pour passer au point suivant. Chacune des directions est codée par 1 chiffre (de 0 à 3 ou de 0 à 7).



Code obtenu : 11007764606544133442

Code minimal (obtenu par rotations) : 00776460654413344211

Il est possible de compresser le code en comptant les occurrences successives : par exemple
0000007777777777 devient 6097 (6 zéros puis 9 septs).

Propriétés utiles pour la reconnaissance

- **Agrandissement** ($\times k$) du contour en répétant k fois chaque descripteur.
- **Rotation** de $k \times 2\pi/n$ (n -connexité) en ajoutant ou retranchant k modulo n à la chaîne initiale.
- **Longueur du descripteur** :
 - en 4 connexité : nb. de descripteurs
 - en 8 connexité : nb descripteurs pairs + $\sqrt{2}$ nb descripteurs impairs.
- **Inversion d'un chemin** : inversion des descripteurs ($j_{inv} = n/2 + j$ modulo n) puis inversion de la séquence.
- **Simplification d'un chemin** : remplacer p descripteurs consécutifs par des descripteurs équivalents reliant les mêmes points.

Moments

Moments géométriques

$$M_{pq} = \sum_x \sum_y x^p y^q I(x, y) \text{ avec } I(x, y) \text{ image au pixel } (x, y)$$

Aire : M_{00} . Centre d'inertie : $\bar{x} = \frac{M_{10}}{M_{00}}, \bar{y} = \frac{M_{01}}{M_{00}}$.

Moments géométriques centrés **invariants par translation**

$$\mu_{pq} = \sum_x \sum_y (x - \bar{x})^p (y - \bar{y})^q I(x, y)$$

$\mu_{00} = M_{00}$ (aire)

μ_{20} et μ_{02} : variances suivant x et y

μ_{30} et μ_{03} : asymétrie (skewness)

μ_{40} et μ_{04} : kurtosis, aplatissement

Moments géométriques centrés normalisés **invariants par translation + échelle**

$$\eta_{pq} = \frac{\mu_{pq}}{\mu_{00}^{\frac{(p+q)}{2} + 1}}$$

Moments



mu20	1,03E+10	1,02E+10	3,55E+08	8,37E+09	1,02E+10
mu11	-3,66E+05	1,04E+06	5,61E+04	1,66E+09	1,04E+06
mu02	6,86E+09	6,86E+09	2,43E+08	8,86E+09	6,86E+09
mu30	5,82E+10	-5,82E+10	-9,07E+08	1,32E+11	-5,82E+10
mu21	-1,31E+11	1,31E+11	1,99E+09	-8,37E+10	1,31E+11
mu12	-2,65E+07	3,87E+07	-7,02E+06	4,84E+10	3,87E+07
mu03	5,82E+10	-5,82E+10	-9,07E+08	1,32E+11	-5,82E+10
nu20	4,13E-04	4,13E-04	4,09E-04	3,35E-04	4,13E-04
nu11	-1,47E-08	4,21E-08	6,46E-08	6,66E-05	4,21E-08
nu02	2,76E-04	2,77E-04	2,80E-04	3,54E-04	2,77E-04
nu30	3,14E-10	1,20E-09	4,96E-10	-1,85E-06	1,20E-09
nu21	-2,36E-06	2,37E-06	2,38E-06	-1,50E-06	2,37E-06
nu12	-4,77E-10	7,00E-10	-8,37E-09	8,66E-07	7,00E-10
nu03	1,05E-06	-1,05E-06	-1,08E-06	2,37E-06	-1,05E-06

6,87E+09	1,65E+09	2,39E+09	1,51E+09	1,47E+09
-2,01E+06	6,05E+06	6,63E+05	1,40E+06	8,33E+05
1,03E+10	1,65E+09	2,39E+09	1,48E+09	1,45E+09
1,11E+08	3,75E+07	6,57E+05	-2,29E+10	2,22E+10
-4,84E+07	-3,74E+07	-2,23E+07	2,33E+10	-2,26E+10
-1,32E+11	-2,82E+07	2,23E+07	1,66E+07	1,43E+07
1,11E+08	3,75E+07	6,57E+05	-2,29E+10	2,22E+10
2,76E-04	3,12E-04	3,27E-04	3,80E-04	3,81E-04
-8,10E-08	1,15E-06	9,07E-08	3,53E-07	2,15E-07
4,14E-04	3,12E-04	3,27E-04	3,74E-04	3,74E-04
1,05E-06	3,46E-09	-5,46E-11	1,38E-08	-7,07E-09
-8,72E-10	-4,66E-09	-1,86E-09	4,15E-06	-4,16E-06
-2,37E-06	-3,52E-09	1,86E-09	2,96E-09	2,64E-09
2,01E-09	4,67E-09	5,46E-11	-4,08E-06	4,08E-06

Moments de Hu¹

7 combinaisons d'invariants centrés définis de façon à être invariants vis-à-vis de la translation, ou de l'échelle, ou de la rotation.

$$h_0 = \eta_{20} + \eta_{02}$$

$$h_1 = (\eta_{20} - \eta_{02})^2 + 4\eta_{11}^2$$

$$h_2 = (\eta_{30} - 3\eta_{12})^2 + (3\eta_{21} - \eta_{03})^2$$

$$h_3 = (\eta_{30} + \eta_{12})^2 + (\eta_{21} + \eta_{03})^2$$

$$h_4 = (\eta_{30} - 3\eta_{12})(\eta_{30} + \eta_{12}) [(\eta_{30} + \eta_{12})^2 - 3(\eta_{21} + \eta_{03})^2] \\ + (3\eta_{21} - \eta_{03}) [3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2]$$

$$h_5 = (\eta_{20} - \eta_{02}) [(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2] + 4\eta_{11}(\eta_{30} + \eta_{12})(\eta_{21} + \eta_{03})$$

$$h_6 = (3\eta_{21} - \eta_{03})(\eta_{30} + \eta_{12}) [(\eta_{30} + \eta_{12})^2 - 3(\eta_{21} + \eta_{03})^2] \\ - (\eta_{30} - 3\eta_{12})(\eta_{21} + \eta_{03}) [3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2]$$

1. M. K. Hu, Visual Pattern Recognition by Moment Invariants, IRE Trans. Info. Theory, vol. IT-8, pp.179-187, 1962

Moments de Hu : exemples



Hu[1]	7,28	7,28	7,28	7,28	7,28	6,70	7,38	7,33	7,19	7,19
Hu[2]	17,79	17,80	17,90	17,83	17,78	13,77	25,95	31,05	23,77	23,67
Hu[3]	23,44	23,44	23,42	23,43	23,43	22,67	35,15	37,30	22,02	22,02
Hu[4]	27,08	27,09	27,11	27,09	27,08	25,03	47,14	39,58	32,96	32,67
Hu[5]	-52,35	-52,35	-52,38	-52,34	-52,34	-48,88	-88,33	78,01	-60,74	-60,02
Hu[6]	-35,98	-35,99	-36,06	-36,00	-35,97	-31,92	-60,77	-55,10	-45,03	-44,52
Hu[7]	-59,79	-57,76	56,22	-56,43	-58,54	-54,27	89,44	-101,28	-60,86	-61,82

Squelette morphologique

Squelette morphologique (morphologie mathématique)

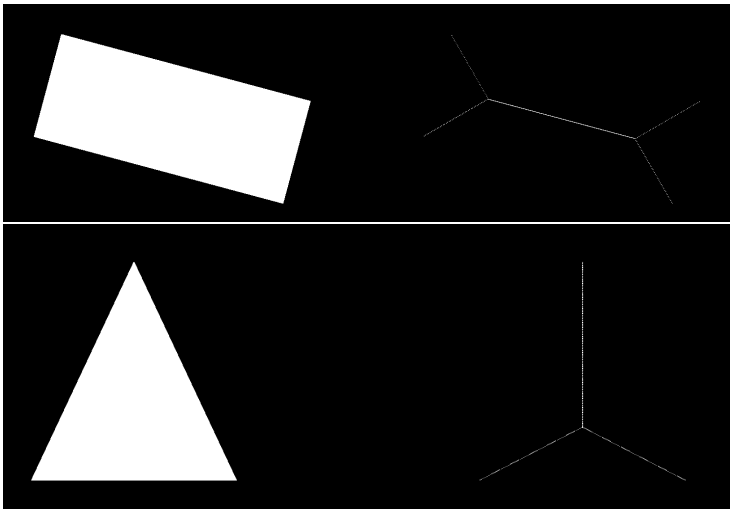
Union des centres des **boules maximales** (plus grandes boules inscrites dans l'objet X)

- B^R : Élément structurant boule de rayon R (carré de taille $(2R + 1) \times (2R + 1)$)
- B^1 : idem, avec rayon $R=1$
- Pour R croissants :
 - $E = \text{erosion}(X, B^R)$
 - $O = \text{dilatation}(E, B^1)$
 - Squelette = ensemble des points appartenant à E et n'appartenant pas à O :
$$S = \{x \in X \mid x \in E \text{ and } x \notin O\}$$
- Arrêt de l'algorithme quand $2R+1 > \text{épaisseur max de } X$

Paramètres de forme

Nombre d'extrémités, d'intersections, de branches, types d'intersections

Squelette (ex squelette morphologique)



TP1 : reconnaissance de formes simples

Structure du programme TP1_RecoFormes.py

- Lecture des images im/formes/00-26.png (avec possibilité de réduire la taille)
 - images 00 à 04 → images d'exemple pour chaque catégorie de forme (0 : cercle, 1 : triangle, 2 : octogone, 3 : carre, 4 : rectangle)
 - images 05 à 26 → images de test
- Binarisation
- Recherche du contour et analyse en composantes connexes (objets). Ici il n'y a qu'un objet par image.
- Calcul de paramètres de formes pour chaque objet par myShapeCompute
- Dans myShapeAnalysis, les paramètres de formes sont comparés à ceux des 5 exemples.
 - renvoie la catégorie de l'objet (entre 0 et 4)
 - compare les résultats avec la vérité terrain contenue dans la liste gt définie au début du programme.

Exo 1.1 : compacité

Fonction myShapeCompute (dans shape.py)

Elle prend en entrée la forme à reconnaître et calcule plusieurs paramètres.

Ajouter les différents paramètres suivants

- **aire** : compter le nombre de pixels de l'objet
- **périmètre** : longueur du contour
 - compléter myFreemanCode pour déduire le code de Freeman de la forme à partir de la chaîne du contour obtenue par cv2.findContours
 - compléter myPerimeter pour en déduire le périmètre
- en déduire la **compacité** et vérifier qu'elle est maximale pour les ronds

Fonction myCompacityAnalysis(param_test, param_ex)

Compléter la fonction pour qu'elle :

- compare la compacité de chaque forme (param_test) avec celles des exemples (param_ex)
- renvoie la catégorie pour chaque forme (0 : cercle, 1 : triangle, 2 : octogone, 3 : carre, 4 : rectangle)

Commenter les résultats obtenus.

Exo 1.2 : moments de Hu

Calcul des moments de Hu

Dans la fonction `myShapeCompute` (dans `shape.py`), rajouter le calcul des moments de Hu (en utilisant les fonctions d'opencv)

Fonction `myHuMomentsAnalysis(param_test, param_ex)`

Compléter la fonction pour qu'elle :

- compare les 7 moments de Hu d'une forme (`param_test`) avec ceux des formes exemples (`param_ex`)
- renvoie la catégorie de la forme

Commenter les résultats obtenus.

Exo 1.3 : signature

Fonction `mySignature`

Compléter la fonction pour qu'elle renvoie la signature de l'objet

Fonction `mySignatureAnalysis(param_test)`

- Mettre à jour la fonction pour qu'elle analyse la signature et en déduise la classe de chaque forme
- Commenter les résultats

Application aux panneaux routiers

- Observer le programme `TP1_RecoPanneaux.py` et décrire les différents traitements réalisés
- Importer votre fonction `mySignatureAnalysis(param_test)` et la modifier pour qu'elle reconnaisse les différentes formes de panneaux : rond, triangle, octogone.